

AI Engineering Boot Camp

Python Foundations — Days 1 to 5

Reference Document

Target: Google · Meta · OpenAI · Anthropic

Boot Camp Rules

These rules apply every single session. Breaking them slows you down.

- Never copy-paste code. Type everything by hand. Your fingers need to know the syntax.
- Every concept must be implemented within 24 hours of learning it.
- Every file must pass `mypy --strict` before it is done.
- Push to GitHub every single day — no exceptions.
- Sunday is complete rest. No code, no videos, no Anki.
- Exit criteria are gates, not suggestions. Do not move forward until you pass them.
- Hard problem slots are sacred — 1 hour, genuine difficulty, struggle for 30 mins before hints.
- Read one paper per week minimum. Write a 3-sentence summary.

The daily workflow for every file:

```
write code → ruff (clean?) → mypy --strict (types correct?) → pytest (works?)  
→ git push
```

Day 1 — Python Fundamentals

Files created

- day1_basics.py — variables and type hints
- day1_strings.py — string methods
- day1_functions.py — functions
- day1_data_structures.py — lists and dictionaries
- day1_loops.py — for, while, enumerate

Type Hints

Type hints are annotations that tell Python and mypy what type a variable or function uses. They are not enforced at runtime — mypy checks them before the code runs.

```
name: str = "Tommy"
age: int = 25
rate: float = 0.001
active: bool = True
```

Function type hints:

```
def add(x: int, y: int) -> int:
    return x + y
```

```
def greet(name: str) -> str:
    return f"Hello {name}"
```

```
def log(msg: str) -> None:    # returns nothing
    print(msg)
```

-> None means the function returns nothing. -> int means it returns an integer.

String Methods

Method	What it does	Example
.upper()	Converts all characters to uppercase	"hello".upper() → "HELLO"
.lower()	Converts all characters to lowercase	"HELLO".lower() → "hello"

.find(x)	Returns index of first char of x	"machine".find("chi") → 2
.split(x)	Splits string into list by delimiter	"a b c".split(" ") → ["a", "b", "c"]
.join(list)	Joins list into one string	" ".join(["a", "b"]) → "a b"
.strip()	Removes whitespace from both ends	" hi ".strip() → "hi"
.replace(a,b)	Replaces all occurrences of a with b	"cat".replace("c", "b") → "bat"
.count(x)	Counts occurrences of x	"hello".count("l") → 2

Lists vs Dictionaries

Lists — access by position (index):

```
scores: list[int] = [95, 87, 92]
scores[0]    # 95 — first item
scores[-1]   # 92 — last item
scores[0:2]  # [95, 87] — slice, end excluded
```

Dictionaries — access by label (key):

```
model: dict[str, str] = {"name": "GPT", "type": "transformer"}
model["name"]         # "GPT"
model.keys()          # all keys
model.values()         # all values
model.items()          # key-value pairs for looping
```

Loops

```
for num in numbers:      # loop over collection
for i in range(5):       # 0, 1, 2, 3, 4
for i in range(1, 6):    # 1, 2, 3, 4, 5
for i in range(0, 10, 2): # 0, 2, 4, 6, 8 (step)
for i, num in enumerate(numbers): # index + value
for key, value in dict.items():   # dict loop
```

```
count: int = 0
while count < 5:           # loop until condition false
    count += 1
```

Day 2 — Functions, Classes, Errors

Files created

- day2_comprehensions.py — list comprehensions
- day2_functions.py — *args, **kwargs, multiple returns
- day2_errors.py — try/except/finally
- day2_classes.py — classes, objects, inheritance

List Comprehensions

One-line loop that builds a list. Pattern: [do_something(item) for item in collection]

```
doubled = [num * 2 for num in numbers]
evens   = [num for num in numbers if num % 2 == 0]
squares = [num ** 2 for num in numbers if num % 2 != 0]
letters = [char.upper() for char in sentence if char != " "]
```

% is modulo — gives remainder after division. 4 % 2 == 0 means even. 5 % 2 == 1 means odd.

Advanced Functions

Default arguments:

```
def greet(name: str, role: str = "engineer") -> str:
    return f"Hello {name}, you are a {role}"
```

Multiple return values (returns a tuple):

```
def get_stats(nums: list[int]) -> tuple[int, int, int]:
    return min(nums), max(nums), sum(nums)
low, high, total = get_stats([1,2,3,4,5])
```

*args — any number of positional arguments (packed as tuple):

```
def add(*numbers: int) -> int:
    return sum(numbers)
add(1, 2)          # works
add(1, 2, 3, 4)    # also works
```

**kwargs — any number of named arguments (packed as dict):

```
def print_config(**settings: str) -> None:
```

```
for key, value in settings.items():
    print(f"{key}: {value}")
print_config(model="GPT", device="cuda")
```

Error Handling

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("cannot divide by zero")
finally:
    print("always runs — error or not")
```

Common errors: `ZeroDivisionError`, `FileNotFoundError`, `KeyError`, `ValueError`, `TypeError`
finally — always runs, used for cleanup (closing files, freeing GPU memory, saving checkpoints).

Classes and OOP

```
class Model:
    def __init__(self, name: str, layers: int) -> None:
        self.name = name          # self = this object
        self.layers = layers

    def describe(self) -> str:
        return f"{self.name} — {self.layers} layers"

# Inheritance
class TransformerModel(Model):
    def __init__(self, name: str, layers: int, heads: int) -> None:
        super().__init__(name, layers)  # calls parent __init__
        self.heads = heads
```

`__init__` is the constructor — runs when object is created.

`self` is like Java's `this` — refers to the current object.

`super().__init__()` calls the parent class constructor.

Day 3 — Python Internals

Files created

- day3_decorators.py — timer, logger decorators
- day3_generators.py — yield, data loading
- day3_context_managers.py — with statement, contextlib
- day3_lambdas.py — lambda, map, filter

Decorators

A function that wraps another function to add behaviour without changing it.

```
from functools import wraps
from typing import Callable, TypeVar, Any

F = TypeVar("F", bound=Callable[..., Any])

def timer(func: F) -> F:
    @wraps(func)                # preserves original function name
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"took {end-start:.4f}s")
        return result
    return wrapper # type: ignore[return-value]

@timer
def train() -> None:
    time.sleep(0.5)
```

`@wraps(func)` preserves the function's `__name__` and metadata after wrapping.
Stacked decorators apply bottom-up. Top decorator runs first (outermost wrapper).

Generators

Produces values one at a time on demand. Uses `yield` instead of `return`.

```
from typing import Generator
```

```
def count_up(n: int) -> Generator[int, None, None]:
    for i in range(n):
        yield i    # pause here, resume when next() called
```

```
def data_loader(dataset: list[int], batch_size: int)
    -> Generator[list[int], None, None]:
    for i in range(0, len(dataset), batch_size):
        yield dataset[i:i + batch_size]
```

yield vs return: yield pauses and resumes. return stops the function permanently.

In ML: DataLoader uses generators to load batches without loading all data into RAM.

Context Managers

Guarantees setup and cleanup always run, even if an error occurs.

```
from contextlib import contextmanager
```

```
@contextmanager
def timer(name: str) -> Generator[None, None, None]:
    start = time.time()
    try:
        yield
    finally:
        print(f"{name}: {time.time()-start:.4f}s")
```

```
with timer("training"):
    train_model()
```

In ML: torch.no_grad(), torch.cuda.amp.autocast() are context managers.

Lambda, map, filter

```
square = lambda x: x ** 2    # one-line function
```

```
# filter - keeps items where condition is True
evens = list(filter(lambda x: x % 2 == 0, numbers))
```

```
# map - transforms every item
doubled = list(map(lambda x: x * 2, numbers))
```



```
# sort by second element of tuple  
models.sort(key=lambda m: m[1])
```

Day 4 — Concurrency

Files created

- day4_threading.py — threading for I/O bound tasks
- day4_multiprocessing.py — multiprocessing for CPU bound tasks
- day4_async.py — async/await event loop

The GIL

Global Interpreter Lock — allows only one Python thread to execute at a time.

Approach	Use for	Why
threading	I/O bound tasks	Downloading, file reading, API calls — GIL releases during waiting
multiprocessing	CPU bound tasks	Math, computation — each process has its own GIL
async/await	I/O bound, many tasks	One thread, switches between tasks during await

Neural network training is CPU/GPU bound — threading does NOT speed it up. Use multiprocessing or GPU (which bypasses the GIL entirely).

Threading

```
import threading
t = threading.Thread(target=download, args=(file_id,))
t.start()
t.join() # wait for thread to finish
```

Multiprocessing

```
import multiprocessing
if __name__ == "__main__": # required on Windows
    with multiprocessing.Pool(processes=4) as pool:
        pool.map(heavy_computation, [5_000_000] * 4)
```

On Windows, multiprocessing code must always be inside `if __name__ == "__main__":`.

Async / Await

```
import asyncio
```

```
async def download(file_id: int) -> str:
    await asyncio.sleep(1)  # pause, let other tasks run
    return f"file_{file_id}"
```

```
async def main() -> None:
    results = await asyncio.gather(
        download(1),
        download(2),
        download(3)
    )
```

```
asyncio.run(main())
```

`await asyncio.sleep()` — pauses this task only, other tasks run during the wait.

`time.sleep()` — blocks the entire thread, nothing else runs.

`asyncio.gather()` — runs multiple async tasks simultaneously.

Day 5 — Type System, Dataclasses, Pydantic

Files created

- day5_types.py — TypeVar, Protocol, dataclasses
- day5_pydantic.py — Pydantic validation and nested models

TypeVar

A placeholder for a flexible but consistent type. The type is determined when the function is called.

```
from typing import TypeVar
T = TypeVar("T")
```

```
def first_item(items: list[T]) -> T:
    return items[0]
```

```
first_item([1, 2, 3])          # T = int, returns int
first_item(["GPT", "BERT"])    # T = str, returns str
```

Protocol

Defines what methods an object must have, without requiring explicit inheritance. Structural subtyping.

```
from typing import Protocol
```

```
class Trainable(Protocol):
    def fit(self, data: list[float]) -> None: ...
    def predict(self, x: float) -> float: ...
```

```
# Any class with fit() and predict() satisfies Trainable
# Even if it never inherits from Trainable
```

Dataclasses

Auto-generates `__init__` from field annotations. Less boilerplate than regular classes.

```
from dataclasses import dataclass, field
```

```
@dataclass
class TrainingConfig:
    learning_rate: float
    batch_size: int
    epochs: int
    model_name: str = "default"
    tags: list[str] = field(default_factory=list)
```

```
def __post_init__(self) -> None:
    if self.learning_rate <= 0:
        raise ValueError("lr must be positive")
```

`__post_init__` runs after `__init__` — used for validation.

`field(default_factory=list)` is required for mutable defaults like lists. Never use `tags: list[str] = []`.

Pydantic

Like dataclasses but with automatic type validation and conversion.

```
from pydantic import BaseModel, field_validator
```

```
class TrainingConfig(BaseModel):
    learning_rate: float
    batch_size: int
    epochs: int
```

```
@field_validator("learning_rate")
@classmethod
def check_lr(cls, v: float) -> float:
    if v <= 0:
        raise ValueError("must be positive")
    return v
```

Pydantic auto-converts types — passing "0.001" as a string becomes 0.001 float.

Use Pydantic for external data (API requests, config files). Use dataclasses for internal data.

	Dataclass	Pydantic BaseModel
Auto <code>__init__</code>	Yes	Yes
Type validation	No	Yes — raises error
Type conversion	No	Yes — "0.001" → 0.001

Use for	Internal data	External data / APIs
---------	---------------	----------------------

Quick Reference — Commands

Daily workflow

```
python filename.py      # run the file
mypy filename.py --strict # type check
ruff check filename.py   # lint
pytest                  # run tests
```

```
git add filename.py
git commit -m "description"
git push
```

Setup

```
pip install ruff mypy pytest aiohttp pydantic
```

File creation (Windows PowerShell)

```
New-Item "filename.py"
Get-ChildItem          # list files
cd D:\PythonWork      # change directory
```

GitHub — first time

```
git init
git add .
git commit -m "first commit"
git remote add origin https://github.com/username/repo.git
git branch -M main
git push -u origin main
```

GitHub — every day after

```
git add .
git commit -m "what you did today"
git push
```

All Anki Cards — Days 1 to 5

Day 1 — Fundamentals

- Q: What does `.upper()` do? A: Converts string to uppercase
- Q: What does `.find()` return? A: Index position of first character of the word
- Q: What does `.split()` return? A: A list of strings
- Q: What does `.join()` do? A: Combines a list into one string
- Q: What does `-> None` mean? A: Function returns nothing
- Q: What does `-> int` mean? A: Function returns an integer
- Q: Difference between list and dict? A: List = position, dict = label
- Q: What does `dict.keys()` return? A: All the keys in the dictionary
- Q: What does `scores[-1]` return? A: The last item in the list
- Q: What does `mypy --strict` do? A: Checks all type hints are correct and consistent
- Q: What does `range(5)` produce? A: 0, 1, 2, 3, 4
- Q: What does `enumerate()` give you? A: Index and value together
- Q: Difference between `for` and `while`? A: `for` = collection, `while` = condition
- Q: What replaces curly braces in Python? A: Indentation

Day 2 — Functions, Classes, Errors

- Q: What is `*args`? A: Accepts any number of positional arguments as a tuple
- Q: What is `**kwargs`? A: Accepts any number of named arguments as a dictionary
- Q: What does `finally` guarantee? A: Always runs whether error occurred or not
- Q: What is `self` in a class? A: Reference to the current object
- Q: What does `super().__init__()` do? A: Calls the parent class constructor
- Q: What is a list comprehension? A: `[do_something(item) for item in collection]`

Day 3 — Internals

- Q: What does `yield` do? A: Pauses function and produces one value, resumes when `next()` called
- Q: What is a generator used for in ML? A: Loading large datasets one batch at a time
- Q: What does a context manager guarantee? A: Setup and cleanup always run
- Q: Difference between `filter()` and `map()`? A: `filter` selects, `map` transforms
- Q: What does `@wraps(func)` do? A: Preserves the original function's name and metadata
- Q: What does `map()` do? A: Applies a function to every item, returns transformed results

- Q: What is a lambda? A: A one-line throwaway function
- Q: What is `_` used for in a for loop? A: Convention for a variable you don't need

Day 4 — Concurrency

- Q: What is the GIL? A: Allows only one Python thread to run at a time
- Q: When does threading help in Python? A: I/O bound tasks — downloading, file reading, API calls
- Q: When does multiprocessing help? A: CPU bound tasks — math, computation
- Q: What does `asyncio.gather()` do? A: Runs multiple async tasks simultaneously
- Q: Difference between `await asyncio.sleep()` and `time.sleep()`? A: `await` pauses just the task, `time.sleep` blocks everything

Day 5 — Type System

- Q: What is `TypeVar` used for? A: A placeholder for a flexible but consistent type
- Q: What is a Protocol? A: Defines what methods an object must have without inheritance
- Q: Difference between `dataclass` and `Pydantic`? A: `Pydantic` validates and converts types, `dataclass` doesn't
- Q: What does `@field_validator` do? A: Adds custom validation logic to a `Pydantic` field
- Q: What does `__post_init__` do in a `dataclass`? A: Runs after `__init__` — used for validation

Next up: Day 6 — NumPy. This is where Python becomes ML engineering.